1. #include <iostream>

```
using namespace std;

// Node structure
class Node {
public:
int data;
Node* next;

Node(int value) {
data = value;
next = NULL;
}
};

// Singly Linked List class
class SinglyLinkedList {
private:
Node* head;

public:
SinglyLinkedList() {
head = NULL;
}

// Insert at end
```

```
void insert(int value) {
Node* newNode = new Node(value);

if (head == NULL) {
head = newNode;
return;
}

Node* temp = head;
while (temp->next != NULL) {
temp = temp->next;
}

temp->next = newNode;
}

// Delete a node by value
void deleteNode(int value) {
if (head == NULL) {
cout << "List is empty!\n";
return;
}

// If head node contains the value
if (head->data == value) {
Node* temp = head;
```

```
head = head->next;
delete temp;
return;
}

Node* current = head;
while (current->next != NULL && current->next->data != value) {
current = current->next;
}

if (current->next == NULL) {
cout << "Value not found!\n";
return;
}

Node* temp = current->next;
current->next = current->next->next;
delete temp;
}

// Display the list
void display() {
Node* temp = head;

while (temp != NULL) {
cout << temp->data << "  ";
```

```
temp = temp->next;
}
cout << "NULL" << endl;
}
};

int main() {
SinglyLinkedList list;

list.insert(10);
list.insert(20);
list.insert(30);

cout << "Linked List: ";
list.display();

list.deleteNode(20);

cout << "After deleting 20: ";
list.display();

return 0;
}
```

Output

```
Linked List: 10 -> 20 -> 30
After deleting 20: 10 -> 30
```

```
1. class Node {
2. public:
3.     int data;
4.     Node* next;
5.
6.     Node(int x) {
7.         data = x;
8.         next = NULL;
9.     }
10. };
11.
12. // Stack class
13. class myStack {
14.
15.
16.    Node* top;
17.
18. public:
19.    myStack() {
20.
21.
22.        top = NULL;
23.    };
```

```
3. include <iostream>
using namespace std;

class Queue {
private:
int arr[100];
int front, rear;

public:
Queue() {
front = -1;
rear = -1;
}

// Insert element into queue
```

```
void enqueue(int value) {
if (rear == 99) {
cout << "Queue Overflow\n";
return;
}

if (front == -1)
front = 0;

arr[++rear] = value;
}

// Delete element from queue
void dequeue() {
if (front == -1 || front > rear) {
cout << "Queue Underflow\n";
return;
}

cout << "Deleted: " << arr[front] << endl;
front++;
}

// Display queue elements
void display() {
if (front == -1 || front > rear) {
cout << "Queue is Empty\n";
return;
}

cout << "Queue Elements: ";
for (int i = front; i <= rear; i++) {
cout << arr[i] << " ";
}
cout << endl;
}
};
```

```
int main() {
Queue q;

q.enqueue(10);
q.enqueue(20);
q.enqueue(30);

q.display();

q.dequeue();

q.display();

return 0;
}
```
Output
Queue Elements: 10 20 30
Deleted: 10
Queue Elements: 20 30
4. include <iostream>
#include <stack>
using namespace std;

class Node {
public:
int data;
Node* next;

Node(int val) {
data = val;
next = NULL;
}
};

// Insert at end
void insert(Node*& head, int val) {
Node* newNode = new Node(val);

```
if (head == NULL) {
head = newNode;
return;
}

Node* temp = head;
while (temp->next != NULL)
temp = temp->next;

temp->next = newNode;
}

// Reverse using stack
void reverseUsingStack(Node* head) {
stack<int> st;
Node* temp = head;

// Push all node data into stack
while (temp != NULL) {
st.push(temp->data);
temp = temp->next;
}

// Replace node data with stack elements
temp = head;
while (temp != NULL) {
temp->data = st.top();
st.pop();
temp = temp->next;
}
}

// Display list
void display(Node* head) {
Node* temp = head;

while (temp != NULL) {
```

```
cout << temp->data << " -> ";
temp = temp->next;
}
cout << "NULL" << endl;
}

int main() {
Node* head = NULL;

insert(head, 10);
insert(head, 20);
insert(head, 30);
insert(head, 40);

cout << "Original List: ";
display(head);

reverseUsingStack(head);

cout << "Reverse List: ";
display(head);

return 0;
}
```

Output

Original List: 10 -> 20 -> 30 -> 40

Reversed List: 40 -> 30 -> 20 -> 10

4.

```
#include <iostream>
#include <stack>
using namespace std;

class TextEditor {
private:
string text;
stack<string> undoStack;
stack<string> redoStack;

public:
```

```
// Perform a new action
void write(string newText) {
undoStack.push(text); // Save current state
text += newText;

// Clear redo history
while (!redoStack.empty())
redoStack.pop();
}

// Undo operation
void undo() {
if (undoStack.empty()) {
cout << "Nothing to undo!\n";
return;
}

redoStack.push(text);
text = undoStack.top();
undoStack.pop();
}

// Redo operation
void redo() {
if (redoStack.empty()) {
cout << "Nothing to redo!\n";
return;
}

undoStack.push(text);
text = redoStack.top();
redoStack.pop();
}

// Display current text
void display() {
cout << "Current Text: " << text << endl;
}
```

```
};

int main() {
TextEditor editor;

editor.write("Hello ");
editor.write("World");
editor.display();

editor.undo();
editor.display();

editor.redo();
editor.display();

return 0;
}
```

Output

```
Current Text: Hello World
Current Text: Hello
Current Text: Hello World
```